

O3 CPU 模型代码分析

概述

`/home/mikasa/0403/gem5/src/cpu/o3` 目录包含完整的 **O3 (Out-of-Order) CPU 模型**，实现了一个乱序超标量处理器。该模型支持 SMT (Simultaneous Multi-Threading)，最多支持 4 个线程。

一、流水线核心阶段

1. `cpu.hh / cpu.cc` - 主 CPU 类

类: **CPU** (继承自 **BaseCPU**)

顶层 CPU 类，协调所有流水线阶段的操作。

主要职责:

- 管理所有流水线阶段 (BAC, Fetch, Decode, Rename, IEW, Commit)
- 协调阶段间通信的时间缓冲
- 处理线程激活/停用和上下文切换
- 管理全局指令序列号
- 提供寄存器文件和重命名映射接口

主要函数:

函数	功能
<code>tick()</code>	主 CPU 周期函数，调用每个阶段的 <code>tick()</code>
<code>drain() / drainResume()</code>	流水线排空，用于检查点
<code>takeOverFrom()</code>	CPU 热切换支持
<code>trap() / processInterrupts()</code>	异常和中断处理
<code>squashFromTC()</code>	外部状态更新导致的 squash
<code>getReg() / setReg()</code>	物理寄存器访问
<code>getArchReg() / setArchReg()</code>	架构寄存器访问

2. `fetch.hh / fetch.cc` - 取指阶段

类: **Fetch**

从 I-cache 取指令并创建动态指令。

主要职责:

- 通过 I-cache 端口从内存取指令
- 管理每线程取指队列

- 集成分支预测器 (通过 BAC/FTQ)
- 支持多种 SMT 取指策略 (轮询、IQ 计数、LSQ 计数、分支计数)
- 管理取指停顿和缓存缺失处理

主要函数:

函数	功能
<code>tick()</code>	取指阶段主周期函数
<code>fetch()</code>	主取指逻辑
<code>fetchCacheLine()</code>	从 I-cache 取缓存行
<code>buildInst()</code>	从静态指令创建动态指令
<code>squashFromCommit()</code>	处理来自提交阶段的 squash
<code>squashFromDecode()</code>	处理来自译码阶段的 squash
<code>getFetchingThread()</code>	基于策略的 SMT 线程选择

内部类:

- `IcachePort` - 指令取指的 RequestPort
- `FetchTranslation` - MMU 转换包装器

3. bac.hh / bac.cc - 分支地址计算阶段

类: `BAC`

处理分支预测和为解耦取指生成获取目标地址。

主要职责:

- 管理分支预测单元 (BPredUnit)
- 为 FTQ 生成取指目标
- 支持解耦前端操作
- 在预测错误时更新分支预测器

主要函数:

函数	功能
<code>tick()</code>	BAC 阶段主周期函数
<code>generateFetchTargets()</code>	通过遍历 BTB 创建取指目标
<code>predict()</code>	分支预测函数
<code>updatePC()</code>	根据指令类型计算下一个 PC
<code>squashBpuHistories()</code>	在 squash 时清除分支预测历史

4. ftq.hh / ftq.cc - 取指目标队列

类: FTQ

在 BAC 和 Fetch 阶段之间保存取指目标的队列。

主要职责:

- 为取指阶段缓冲取指目标
- 维护每个取指目标的分支预测历史
- 支持 squash 和恢复操作

类: FetchTarget

代表单个取指目标 (类似基本块)。

主要函数:

函数	功能
<code>insert()</code>	插入取指目标
<code>readHead()</code> / <code>popHead()</code>	读取/弹出队首目标
<code>squash()</code>	在预测错误时清除取指目标
<code>finalize()</code>	用分支信息完成取指目标

5. decode.hh / decode.cc - 译码阶段

类: Decode

译码已取指令并检查 PC 相关分支。

主要职责:

- 验证 PC 相关分支预测
- 在停顿期间将指令缓冲在滑移缓冲区中
- 处理在译码阶段检测到的分支预测错误

主要函数:

函数	功能
<code>tick()</code>	译码阶段主周期函数
<code>decode()</code>	主译码逻辑
<code>decodeInsts()</code>	处理来自取指的指令
<code>squash()</code>	处理来自提交的 squash
<code>sortInsts()</code>	按线程分离指令 (SMT)

6. rename.hh / rename.cc - 重命名阶段

类: **Rename**

执行寄存器重命名并将指令分发到 IQ/LSQ/ROB。

主要职责:

- 将架构寄存器映射到物理寄存器
- 维护重命名历史用于 squash 恢复
- 处理串行化指令
- 当 ROB/IQ/LSQ 满时阻塞

主要函数:

函数	功能
<code>tick()</code>	重命名阶段主周期函数
<code>rename()</code>	主重命名逻辑
<code>renameInsts()</code>	重命名源/目的寄存器
<code>renameSrcRegs()</code> / <code>renameDestRegs()</code>	寄存器重命名
<code>squash()</code>	在 squash 时撤销上寄存器映射
<code>removeFromHistory()</code>	在提交时清除重命名历史
<code>serializeAfter()</code>	处理串行化依赖

内部结构:

- **RenameHistory** - 跟踪寄存器重命名用于恢复

7. iew.hh / iew.cc - 发射/执行/写回阶段

类: **IEW**

合并阶段，处理指令分发、发射、执行和写回。

主要职责:

- 将指令分发到 IQ 和 LSQ
- IQ 调度就绪指令到功能单元
- 执行非内存指令
- LSQ 执行内存指令
- 写回唤醒依赖指令

主要函数:

函数	功能
<code>tick()</code>	IEW 阶段主周期函数

函数	功能
<code>dispatch()</code>	分发指令到 IQ/LSQ
<code>executeInsts()</code>	执行已调度指令
<code>writebackInsts()</code>	写回结果并唤醒依赖
<code>wakeDependents()</code>	唤醒等待已完成寄存器的指令
<code>squash()</code>	处理 IEW 中的 squash
<code>rescheduleMemInst()</code> / <code>replayMemInst()</code>	内存指令重放

8. commit.hh / commit.cc - 提交阶段

类: `Commit`

从 ROB 按顺序提交指令并处理所有 squash。

主要职责:

- 从 ROB 头部按顺序提交指令
- 处理分支预测错误和陷阱
- 支持多种 SMT 提交策略
- 向所有流水线阶段广播 squash
- 在提交时更新架构状态

主要函数:

函数	功能
<code>tick()</code>	提交阶段主周期函数
<code>commit()</code>	主提交逻辑
<code>commitInsts()</code>	尽可能多地提交指令
<code>commitHead()</code>	提交 ROB 头部指令
<code>getInsts()</code>	从重命名获取指令到 ROB
<code>markCompletedInsts()</code>	标记来自 IEW 的已完成指令
<code>squashFromTrap()</code> / <code>squashFromTC()</code> / <code>squashFromSquashAfter()</code>	squash 处理器
<code>handleInterrupt()</code>	处理中断
<code>getCommittingThread()</code>	SMT 线程选择 (轮询、最旧优先)

二、支持结构

1. inst_queue.hh / inst_queue.cc - 指令队列

类: **InstructionQueue (IQ)**

保存已调度指令并管理依赖。

主要职责:

- 通过依赖图跟踪寄存器依赖
- 调度就绪指令到功能单元
- 通过 MemDepUnit 处理内存指令依赖
- 管理背靠背调度

主要函数:

函数	功能
<code>insert() / insertNonSpec() / insertBarrier()</code>	指令插入
<code>scheduleReadyInsts()</code>	调度操作数就绪的指令
<code>getInstToExecute()</code>	返回下一个要执行的指令
<code>wakeDependents()</code>	唤醒消费者指令
<code>squash()</code>	移除已 squash 的指令
<code>rescheduleMemInst() / replayMemInst()</code>	内存重放

类: **IQUnit**

每线程 IQ 管理，支持 SMT 策略。

内部类:

- `FUCompletion` - FU 执行完成事件

2. `lsq.hh / lsq.cc` - 加载存储队列

类: **LSQ**

管理所有线程的加载和存储队列的顶层 LSQ。

主要职责:

- 执行加载和存储指令
- 跟踪内存顺序违规
- 处理内存端口仲裁
- 管理存储写回内存

主要函数:

函数	功能
<code>insertLoad() / insertStore()</code>	内存指令插入

函数	功能
<code>executeLoad()</code> / <code>executeStore()</code>	内存执行
<code>commitLoads()</code> / <code>commitStores()</code>	提交内存操作
<code>writebackStores()</code>	将存储写回内存
<code>squash()</code>	处理 squash
<code>recvTimingResp()</code>	处理内存响应

类: **DcachePort**

数据缓存访问的 RequestPort。

内部类:

- **LSQRequest** - 内存请求处理基类
- **SingleDataRequest** - 单一数据请求
- **SplitDataRequest** - 分割数据请求
- **UnsquashableDirectRequest** - 不可 squash 的直接请求

3. `lsq_unit.hh` / `lsq_unit.cc` - 每线程 LSQ

类: **LSQUnit**

每线程加载和存储队列实现。

主要职责:

- 实现循环 LQ 和 SQ 缓冲区
- 检查内存顺序违规
- 处理部分加载-存储转发
- 管理存储写回

主要函数:

函数	功能
<code>insertLoad()</code> / <code>insertStore()</code>	插入内存操作
<code>executeLoad()</code> / <code>executeStore()</code>	执行加载/存储
<code>checkViolations()</code>	检查顺序违规
<code>commitLoad()</code> / <code>commitLoads()</code> / <code>commitStores()</code>	提交操作
<code>writebackStores()</code>	写回存储
<code>completeDataAccess()</code>	处理内存完成
<code>recvTimingResp()</code>	内存响应处理器

内部类:

- `LSQEntry` / `LQEntry` / `SQEntry` - 队列条目结构
- `WritebackEvent` - 存储写回事件

4. rob.hh / rob.cc - 重排序缓冲

类: `ROB` (Reorder Buffer)

按顺序提交缓冲，跟踪所有飞行中的指令。

主要职责:

- 为顺序提交缓冲指令
- 驱动 squash 操作
- 管理 SMT ROB 分区

主要函数:

函数	功能
<code>insertInst()</code>	将指令插入 ROB
<code>readHeadInst()</code>	获取 ROB 头部指令
<code>retireHead()</code>	移除已提交的头部指令
<code>squash()</code>	标记指令为已 squash
<code>isHeadReady()</code>	检查头部指令是否可提交
<code>canCommit()</code>	检查是否有线程可提交

三、寄存器管理

1. regfile.hh - 物理寄存器文件

类: `PhysRegFile`

支持多种寄存器类的物理寄存器文件。

主要职责:

- 存储整数、浮点、向量、矩阵、谓词和 CC 寄存器
- 提供寄存器读写访问
- 管理杂项寄存器

主要函数:

函数	功能
<code>getReg()</code> / <code>setReg()</code>	寄存器访问 (多个重载)
<code>getMiscRegId()</code>	获取杂项寄存器 ID

函数	功能
<code>initFreeList()</code>	用寄存器 ID 初始化空闲列表

2. free_list.hh - 空闲列表

类: `UnifiedFreeList`

管理用于重命名的空闲物理寄存器。

主要职责:

- 维护每类空闲寄存器列表
- 为重命名阶段分配空闲寄存器
- 在提交/squash 时接收释放的寄存器

主要函数:

函数	功能
<code>getReg()</code>	分配空闲寄存器
<code>addReg()</code> / <code>addRegs()</code>	将寄存器返回空闲列表
<code>numFreeRegs()</code>	查询空闲寄存器数量

类: `SimpleFreeList`

每寄存器类的空闲列表。

3. rename_map.hh - 重命名映射

类: `UnifiedRenameMap`

架构寄存器到物理寄存器的映射。

主要职责:

- 将架构寄存器映射到物理寄存器
- 通过 `setEntry()` 支持 squash 恢复
- 处理杂项寄存器重映射

主要函数:

函数	功能
<code>rename()</code>	为架构寄存器分配新物理寄存器
<code>lookup()</code>	获取架构寄存器的物理寄存器
<code>setEntry()</code>	更新映射 (用于 squash 恢复)
<code>minFreeEntries()</code>	跨类的最小空闲寄存器

类: SimpleRenameMap

每寄存器类的重命名映射。

4. scoreboard.hh - 记分板

类: Scoreboard

跟踪物理寄存器就绪状态。

主要职责:

- 位向量跟踪就绪的物理寄存器
- 在写回和提交时更新

主要函数: | 函数 | 功能 | |--| | getReg() | 检查寄存器是否就绪 || setReg() | 标记寄存器为就绪 || unsetReg() | 标记寄存器为未就绪 |

四、内存依赖

1. mem_dep_unit.hh / mem_dep_unit.cc - 内存依赖单元

类: MemDepUnit

跟踪内存指令依赖。

主要职责:

- 使用 store sets 预测内存依赖
- 跟踪飞行中的内存操作
- 唤醒依赖的内存指令

主要函数:

函数	功能
insert() / insertNonSpec() / insertBarrier()	插入内存操作
regsReady() / nonSpecInstReady()	标记指令就绪
reschedule() / replay()	内存重放
completeInst()	标记指令完成
violation()	处理顺序违规
wakeDependents()	唤醒等待指令

内部类:

- MemDepEntry - 内存依赖跟踪条目

2. store_set.hh / store_set.cc - Store Set 预测器

类: **StoreSet**

基于 store set 的内存依赖预测器。

主要职责:

- **SSIT** (Store Set ID Table) - 将 PC 映射到 store set ID
- **LFST** (Last Fetched Store Table) - 跟踪每个集合的最后存储
- 预测 load-store 依赖

主要函数:

函数	功能
<code>violation()</code>	记录顺序违规以学习
<code>insertLoad()</code> / <code>insertStore()</code>	插入内存操作操作
<code>checkInst()</code>	检查预测依赖
<code>issued()</code>	记录指令发射
<code>squash()</code>	在 squash 时清除预测

内部类:

- **SSITEntry** - SSIT 缓存条目

3. dep_graph.hh - 依赖图

类: **DependencyGraph<DynInstPtr>**

基于链表的寄存器依赖图。

主要职责:

- 跟踪每个物理寄存器的生产者-消费者关系
- 生产者完成时唤醒消费者

主要函数:

函数	功能
<code>insert()</code>	将消费者添加到依赖列表
<code>setInst()</code> / <code>clearInst()</code>	设置/清除生产者
<code>remove()</code>	从图中移除指令
<code>pop()</code>	移除并返回最新的依赖

内部类:

- **DependencyEntry** - 依赖列表节点

五、功能单元

1. fu_pool.hh / fu_pool.cc - 功能单元池

类: **FUPool**

管理具有各种能力的功能单元。

主要职责:

- 基于操作类为指令分配 FU
- 跟踪 FU 忙闲状态
- 执行后管理 FU 释放

主要函数:

函数	功能
getUnit()	获取操作类的空闲 FU
freeUnitNextCycle()	调度 FU 释放
processFreeUnits()	释放调度的 FU
isCapable()	检查 FU 能力

内部类:

- **FUIdxQueue** - FU 索引的循环队列

六、线程管理

1. thread_context.hh / thread_context.cc - 线程上下文

类: **ThreadContext** (继承自 `gem5::ThreadContext`)

线程状态的外部接口。

主要职责:

- 提供架构状态访问
- 在外部状态更新时触发 squash
- 处理 PC 事件和指令计数事件

主要函数:

函数	功能
pcState()	PC 状态访问

函数	功能
<code>readMiscReg()</code> / <code>setMiscReg()</code>	杂项寄存器访问
<code>getReg()</code> / <code>setReg()</code>	架构寄存器访问
<code>activate()</code> / <code>suspend()</code> / <code>halt()</code>	线程状态控制
<code>takeOverFrom()</code>	线程上下文接管

2. thread_state.hh / thread_state.cc - 线程状态

类: **ThreadState** (继承自 `gem5::ThreadState`)

每线程状态存储。

主要职责:

- 存储线程状态和进程指针
- 管理 PC 事件队列
- 跟踪陷阱待定和 squash 抑制状态

主要成员:

- `pcEventQueue` - 基于 PC 的事件队列
- `comInstEventQueue` - 指令计数事件队列
- `trapPending` - 陷阱处理标志
- `noSquashFromTC` - Squash 抑制标志
- `htmCheckpoint` - HTM 检查点指针

七、通信结构

comm.h - 阶段间通信

结构体:

结构体	用途
<code>FetchStruct</code>	取指到译码通信
<code>DecodeStruct</code>	译码到重命名通信
<code>RenameStruct</code>	重命名到 IEW 通信
<code>IEWStruct</code>	IEW 到提交通信
<code>IssueStruct</code>	发射阶段通信
<code>TimeStruct</code>	反向通信 (停顿、squash、空闲条目)

八、探查/跟踪

1. probe/simple_trace.hh - 简单跟踪

类: **SimpleTrace** (继承自 **ProbeListenerObject**)

简单指令跟踪生成器。

主要函数:

函数	功能
<code>traceFetch()</code> / <code>traceCommit()</code>	跟踪处理器

2. probe/elastic_trace.hh - 弹性跟踪

类: **ElasticTrace** (继承自 **ProbeListenerObject**)

详细的依赖跟踪生成器，用于 TraceCPU 回放。

主要职责:

- 记录指令时序和依赖
- 计算计算延迟
- 生成用于回放的 protobuf 跟踪

内部结构:

- **InstExecInfo** - 临时执行信息存储
- **TraceInfo** - 最终跟踪记录

九、配置

limits.hh

编译时常量:

常量	值	描述
<code>MaxWidth</code>	16	最大流水线宽度
<code>MaxThreads</code>	4	最大 SMT 线程数

十、动态指令

dyn_inst.hh / dyn_inst.cc - 动态指令

类: **DynInst** (继承自 **ExecContext**, **RefCounted**)

代表飞行中的动态指令实例。

主要职责:

- 跟踪指令在流水线中的状态

- 存储已重命名的寄存器映射
- 处理内存访问和错误
- 记录执行结果

主要函数 (状态管理):

函数	功能
<code>setCompleted() / isCompleted()</code>	完成状态
<code>setCanIssue() / readyToIssue()</code>	发射就绪状态
<code>setIssued() / isIssued()</code>	发射跟踪
<code>setExecuted() / isExecuted()</code>	执行状态
<code>setCanCommit() / readyToCommit()</code>	提交就绪状态
<code>setCommitted() / isCommitted()</code>	提交状态
<code>setSquashed() / isSquashed()</code>	Squash 状态
<code>setInIQ() / isInIQ()</code>	IQ 驻留
<code>setInLSQ() / isInLSQ()</code>	LSQ 驻留
<code>setInROB() / isInROB()</code>	ROB 驻留

主要函数 (操作):

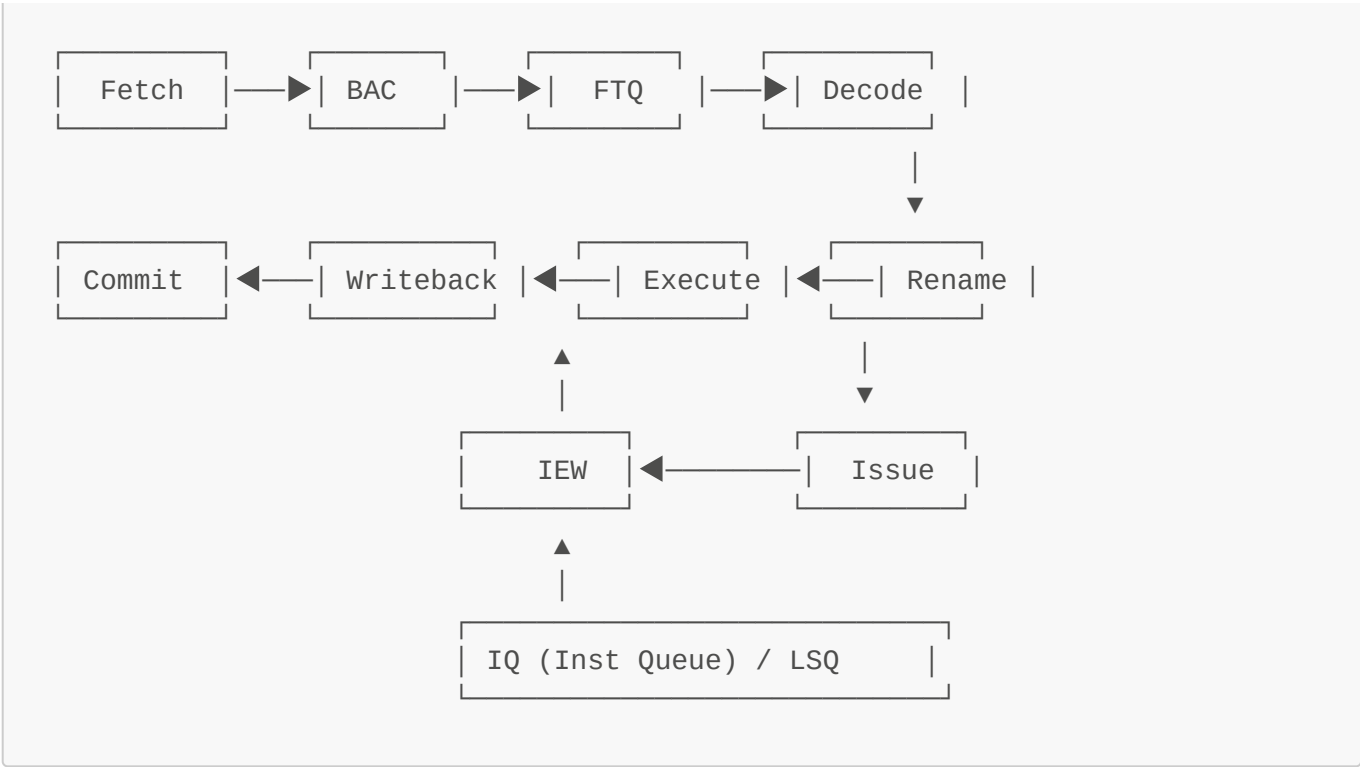
函数	功能
<code>execute()</code>	执行指令
<code>initiateAcc() / completeAcc()</code>	内存访问
<code>renameDestReg() / renameSrcReg()</code>	寄存器重命名访问
<code>getRegOperand() / setRegOperand()</code>	执行期间寄存器访问
<code>readMiscReg() / setMiscReg()</code>	杂项寄存器访问

指令类型查询:

函数	功能
<code>isLoad() / isStore()</code>	加载/存储指令
<code>isControl()</code>	控制指令
<code>isSerializing()</code>	串行化指令

状态位: 广泛的位字段跟踪指令在流水线中的状态

十一、流水线数据流图



十二、关键特性总结

特性	描述
SMT 支持	最多 4 线程同时多线程
可配置共享策略	IQ/LSQ/ROB 的线程间分区策略
内存依赖预测	基于 store sets 的详细预测器
解耦取指	BAC/FTQ 分离分支预测和取指
乱序执行	完整的寄存器重命名和依赖跟踪
精确异常	通过 ROB 实现顺序提交

生成信息

- 生成时间: 2026-04-15
- 分析的目录: /home/mikasa/0403/gem5/src/cpu/o3
- gem5 版本: v25.1.0.0